

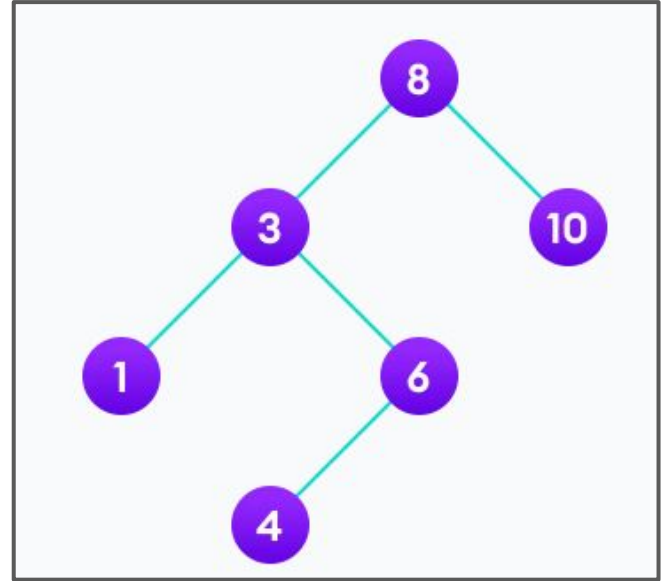
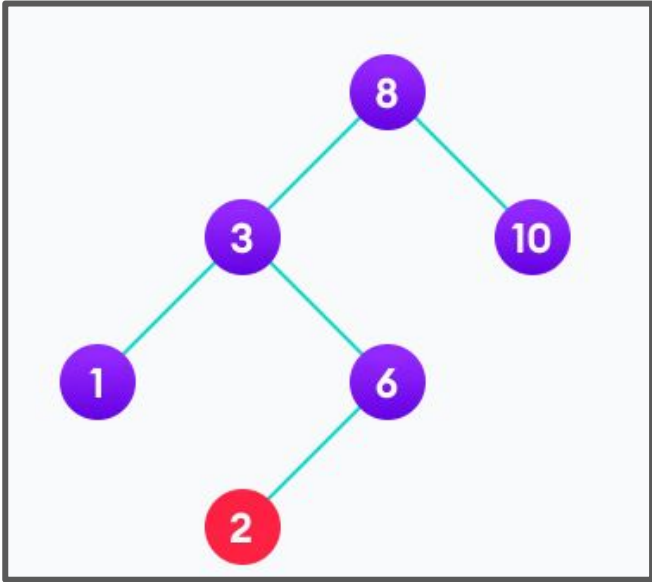
Binary search Tree

PROPERTIES

Binary search tree is a data structure that quickly allows us to maintain a sorted list of numbers.

- **It is called a binary tree because each tree node has a maximum of two children.**
- **All nodes of left subtree are less than the root node**
- **All nodes of right subtree are more than the root node**

EXAMPLE



WHICH ONE IS CORRECT??

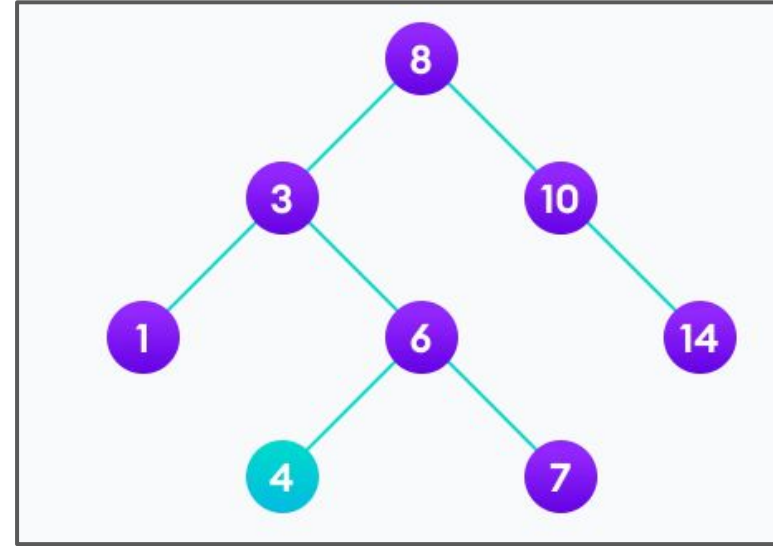
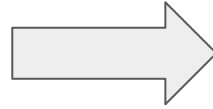
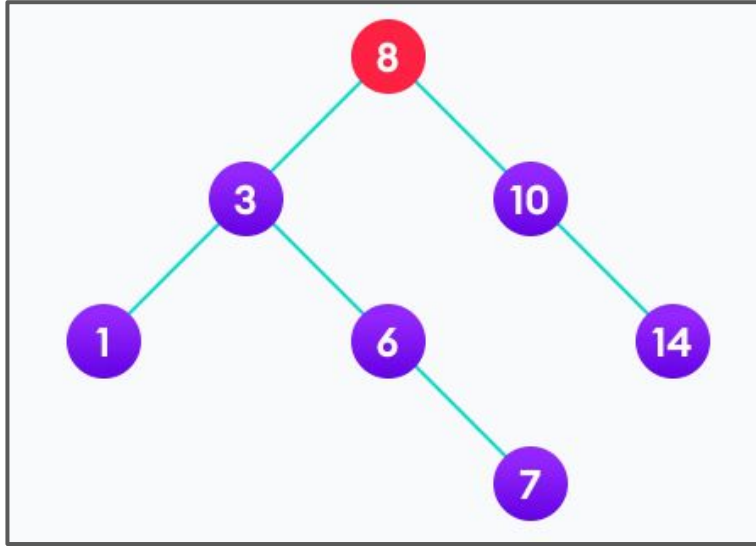
Insert Operation

Inserting a value in the correct position is similar to searching because we try to maintain the rule that

- the left subtree is lesser than root and
- the right subtree is larger than root.

```
if (root == NULL) {  
    return createNode(value);  
}  
  
// Insert into the left subtree  
if (value < root->data) {  
    root->left = insertNode(root->left, value);  
}  
  
// Insert into the right subtree  
else if (value > root->data) {  
    root->right = insertNode(root->right, value);  
}  
  
return root;
```

EXAMPLE



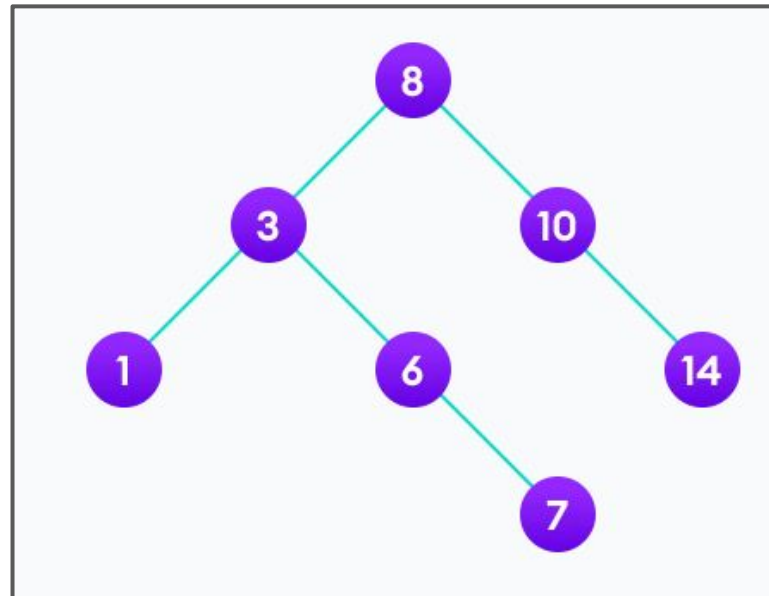
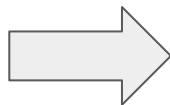
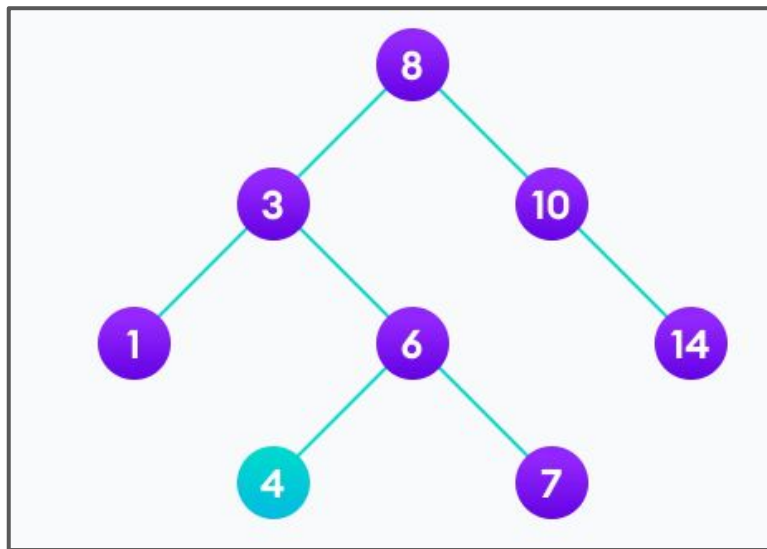
Insert 4 in the BST

Deletion Operation

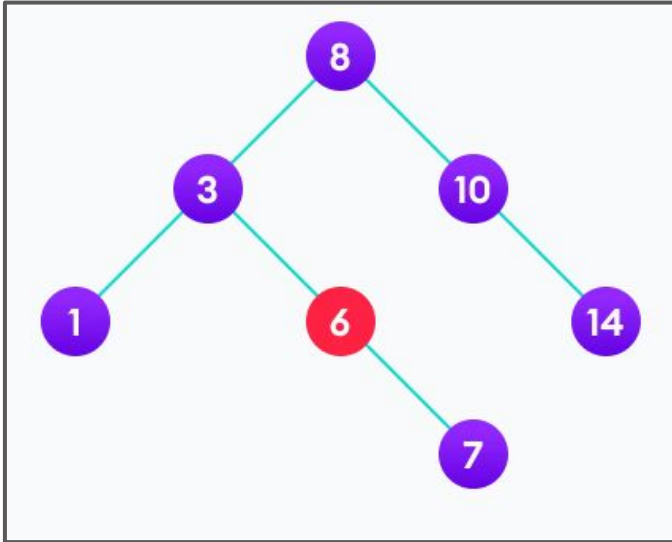
Three cases for deleting a node from a binary search tree.

1. **NODE is a LEAF node.**
2. **DELETED NODE has a SINGLE child node.**
3. **DELETED NODE has TWO child node.**

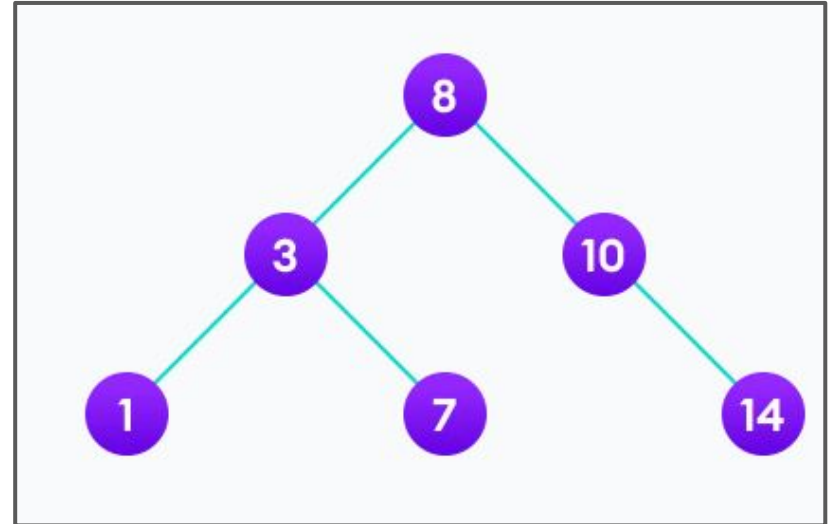
CASE-1



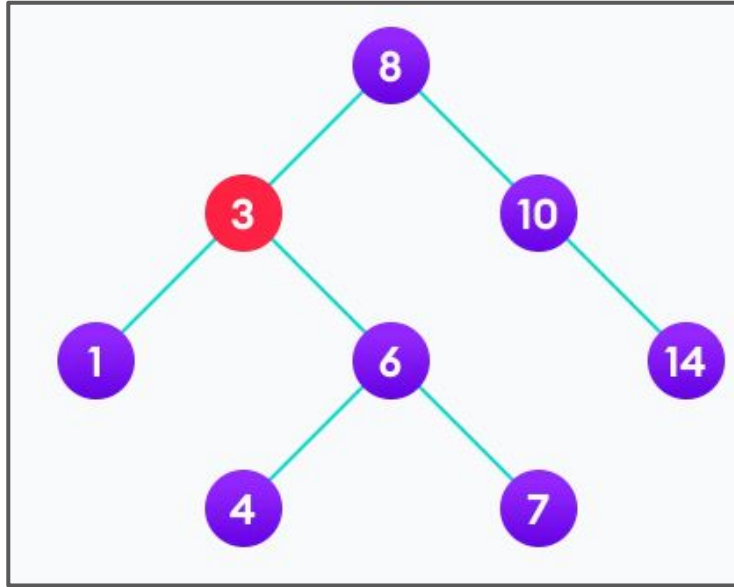
CASE-2



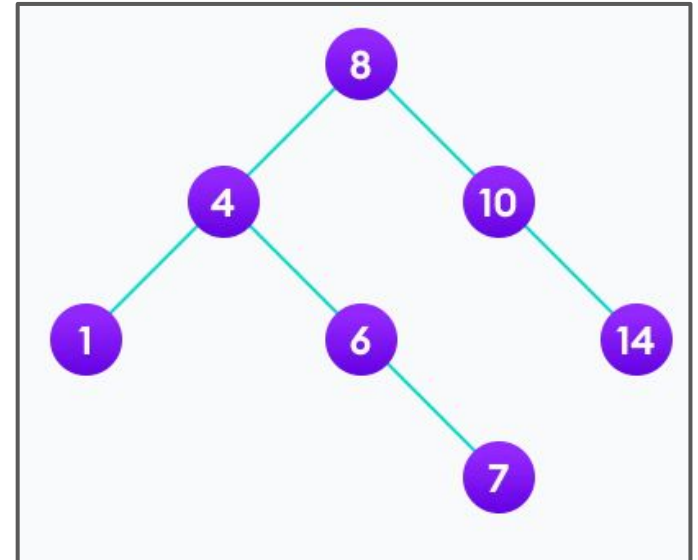
- Replace that node with its child node.
- Remove the child node from its original position.



CASE-3



- Get the inorder successor of that node.
- Replace the node with the inorder successor.
- Remove the inorder successor from its original position.



DELETION CODE

```
// Find the node to be deleted
if (key < root->key)
    root->left = deleteNode(root->left, key);
else if (key > root->key)
    root->right = deleteNode(root->right, key);
```

```
else {
    // If the node is with only one child or no child
    if (root->left == NULL) {
        struct node *temp = root->right;
        free(root);
        return temp;
    } else if (root->right == NULL) {
        struct node *temp = root->left;
        free(root);
        return temp;
    }
}
```

```
// If the node has two children
struct node *temp = minValueNode(root->right);

// Place the inorder successor in position of the
node to be deleted
root->key = temp->key;

// Delete the inorder successor
root->right = deleteNode(root->right, temp->key);
}
return root;
}
```

